

COMS4995W30

Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Fall 2026



Announcement



Mid-term 😲



Week 6

Duration: 1.5 hours

Format:

- Multiple Choice 
- Short Answer 

Cheat Sheet:

- 1-page A4 (double-sided allowed)
- Handwritten or printed

What You Need to Know



Course 1 - 5:

- Data prep & feature engineering
- Generative vs. Discriminative models
- Model evaluation & bias - variance
- Ensemble

Focus on:

- Understanding concepts
- Explaining scenarios
- Sharing intuitions





Model Evaluation

Bias-Variance

AI of the Week: Gemini 4 Argon



Sept 30, 2026: Google announces Gemini 4 Argon (limited release)

- 18 benchmarks, leading on many
- Coding (DeepSWE v1.1): 77.9% vs Claude Opus 5.5 74.2%, GPT-6 Astra 74.1%

The Problem: hill-climbing the benchmark for all advanced LLMs

- Bloomberg report: strong on standard benchmarks, weaker when people use it for real work
- Trick to game the system

 Class Connection: Overfitting

- Tune against the same benchmarks long enough and they stop being a test set

Agenda



- Motivation
- Train / Validation / Test Split
- Common Evaluation Metrics
- Bias - Variance Tradeoff
- Model Selection Strategies



Motivation



Supervised vs. Unsupervised Learning

Supervised Learning

- Learn mapping $f: X \rightarrow Y$
- Given data (x_i, y_i)
- Example: Spam Email (label = spam / ham)

Unsupervised Learning

- Discover hidden structures in data
- Only x_i , no labels
- Example: Customer Segmentation



What is Machine Learning (ML)?

- Learn patterns from **data** 
- Make **predictions** on new data 
- **Generalize** beyond training 

Why Do We Need Model Evaluation? 🤔

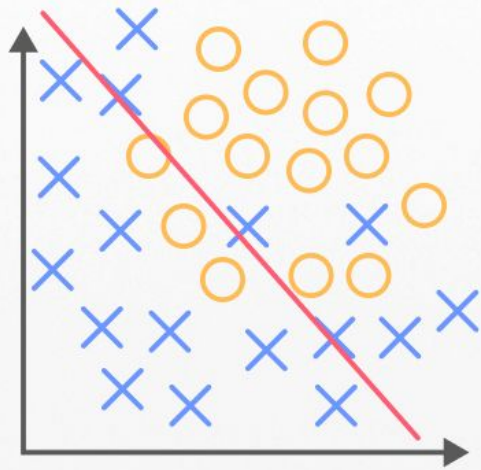


Training accuracy $\neq$ Real-world performance

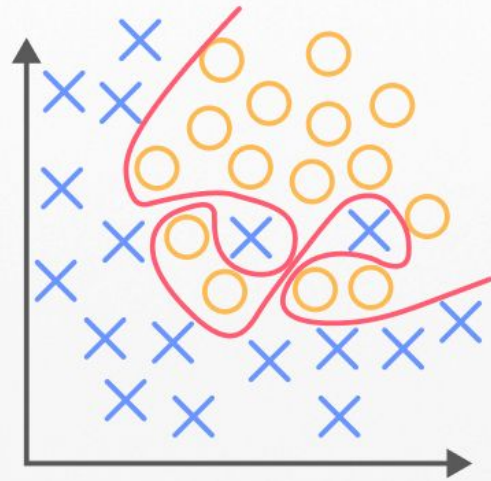
Without proper evaluation:

- Models may memorize training data (**overfitting**)
- Models may be too simple to capture patterns (**underfitting**)

Wrong evaluation $\rightarrow$ Wrong decisions $\rightarrow$ Costly mistakes 💰

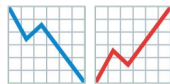


Underfitting



Overfitting

Overfitting vs Underfitting



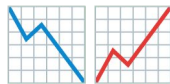
Overfitting definition:

- Model fits the training data too closely - including random noise
- **Symptom:**
 - Training Accuracy $\approx 100\%$
 - Test Accuracy $\ll$ Training Accuracy

Like a student who:

- 📖 Memorizes every word from lecture slides (training set)
- ❌ Cannot answer slightly different questions in mid-term (test set)

Overfitting vs Underfitting



Underfitting definition:

- Model is too simple to capture the underlying patterns in data.
- **Symptom:**
 - Low Training Accuracy
 - Low Test Accuracy

Like a student who:

📖 Barely skimmed the textbook

✗ Cannot even handle problems in the textbook (training set)

✗ Naturally also fails in mid-term (test set)



Mock Interview



Interviewer: “Your disease-detection model reports 99% accuracy. 1% of patients are actually sick. Do you ship it?”

- A) Yes - 99% is excellent
- B) Yes, once it also gets 99% on the test set
- C) Not yet - “always predict healthy” also gets 99%
- D) No - 99% means it is overfitting

30 seconds, no talking.

Why Wrong Metrics Can Be Dangerous



Accuracy is misleading when data is imbalanced

Example: disease detection dataset

- 99% healthy, 1% sick
- A model that always predicts “healthy” → 99% accuracy but useless 

Need metrics that capture what matters: **precision**, **recall**, etc.

Key Takeaway



- Evaluation is not optional, it is **essential**
- Good evaluation = Fair model comparison
- Good evaluation = Reliable deployment in real world



Train / Validation / Test Split

Why Split Data? 🍰



Goal: measure **generalization ability**

If we only check training accuracy:

- Might think model is “perfect” → but actually memorizing 📖

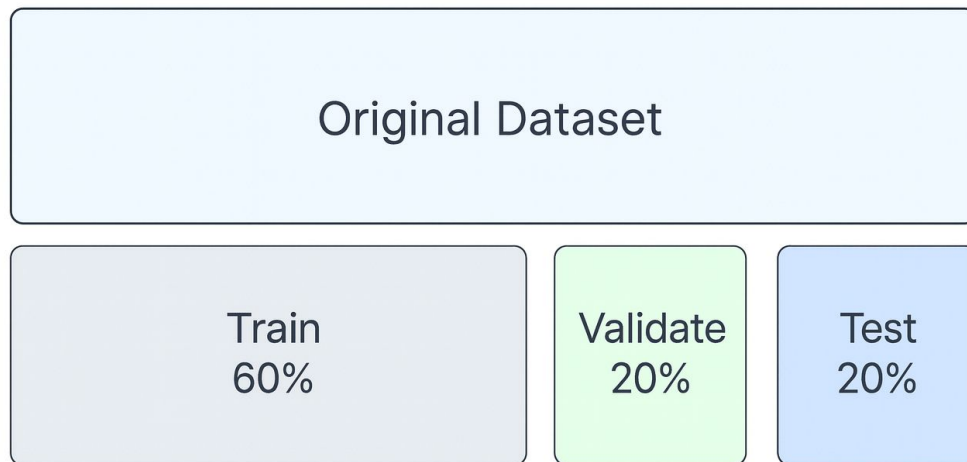
Solution: keep separate sets

- **Train** → learn patterns
- **Validation** → tune hyperparameters
- **Test** → simulate unseen real-world data

Why Split Data? 🍰



Multiple Models Percentage Split



Typical Splits

Common ratios:

- Train: 60 - 80%
- Validation: 10 - 20%
- Test: 10 - 20%

Important rule: Test set is locked 



Hold-Out Method



One-time split (Train / Val / Test)

- Pros: simple, fast
- Cons:
 - depends heavily on the random split
 - unstable on small data

k-Fold Cross-Validation



 Idea: Rotate the validation set

Steps:

- Split into k equal folds
- Train on $(k-1)$ folds, validate on the remaining one
- Repeat k times, average results

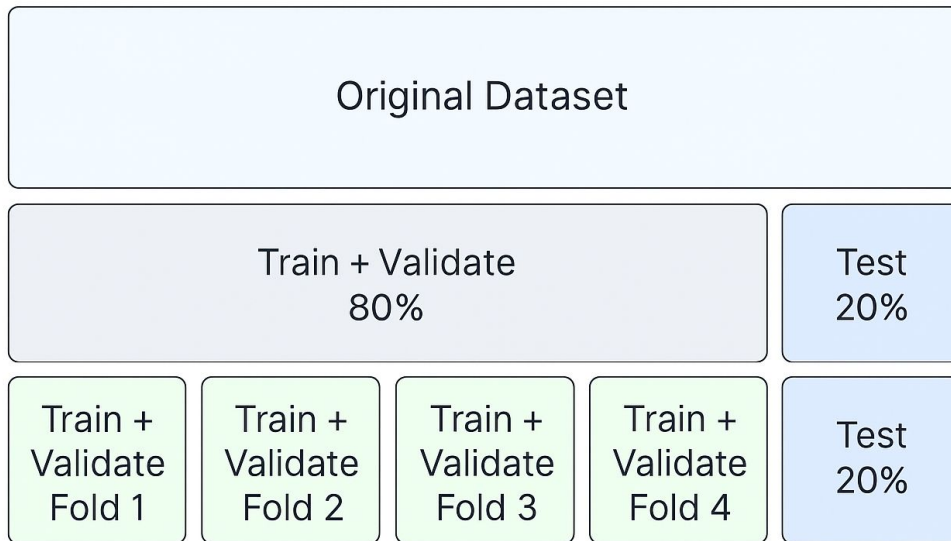
Pros: robust, uses all data

Cons: more compute 

k-Fold Cross-Validation



Multiple Models K-Fold Cross Validation $K = 4$





Common Evaluation Metrics

Why Metrics Matter



Different problems need different metrics

Example:

- Credit card fraud detection → catch rare cases
- Recommender system → focus on ranking quality

 Accuracy is not enough 

Regression Metrics



Mean Squared Error (MSE)

- Penalizes large errors more

$$\text{MSE} = \overset{\text{Mean}}{\frac{1}{n} \sum_{i=1}^n} \left(\overset{\text{Error}}{Y_i - \hat{Y}_i} \right) \overset{\text{Squared}}{^2}$$

Regression Metrics



Mean Absolute Error (MAE)

- Less sensitive to outliers

$$MAE = \frac{1}{n} \sum \left| y - \hat{y} \right|$$

Diagram illustrating the Mean Absolute Error (MAE) formula:

- $\frac{1}{n}$: Divide by the total number of data points
- $\sum$: Sum of
- y : Actual output value
- $\hat{y}$: Predicted output value
- $|y - \hat{y}|$: The absolute value of the residual

Classification Metrics /



Accuracy

- Proportion of correct predictions
- Simple but misleading on imbalanced data

Confusion Matrix

1 2
3 4



		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Recall $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Precision, Recall



Precision (**P**urity)

- Out of **predicted positives**, how many are correct?

Recall (**C**overage)

- Out of **actual positives**, how many did we find?

Example: Disease detection

- High recall = fewer missed cases
- High precision = fewer false alarms

F1



F1 combines Precision and Recall into one metric

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$



Mock Interview



Interviewer: “Cancer screening. We flag a patient when the model score is above 0.5. Missing a sick patient is far worse than a false alarm. What do you do to the threshold?”

- A) Raise it - be more certain before flagging
- B) Lower it - catch more cases, accept more false alarms
- C) Keep 0.5 - it is the natural cutoff
- D) Retrain - the threshold does not matter

30 seconds, no talking.

Different Precision & Recall



A model outputs **scores / probabilities**, not just hard labels

To decide class = 1, we must set a threshold

- Example: predict positive if score ≥ 0.5

If we change the threshold:

- Lower threshold $\rightarrow$ more positives predicted $\rightarrow$ higher Recall, lower Precision
- Higher threshold $\rightarrow$ fewer positives predicted $\rightarrow$ higher Precision, lower Recall

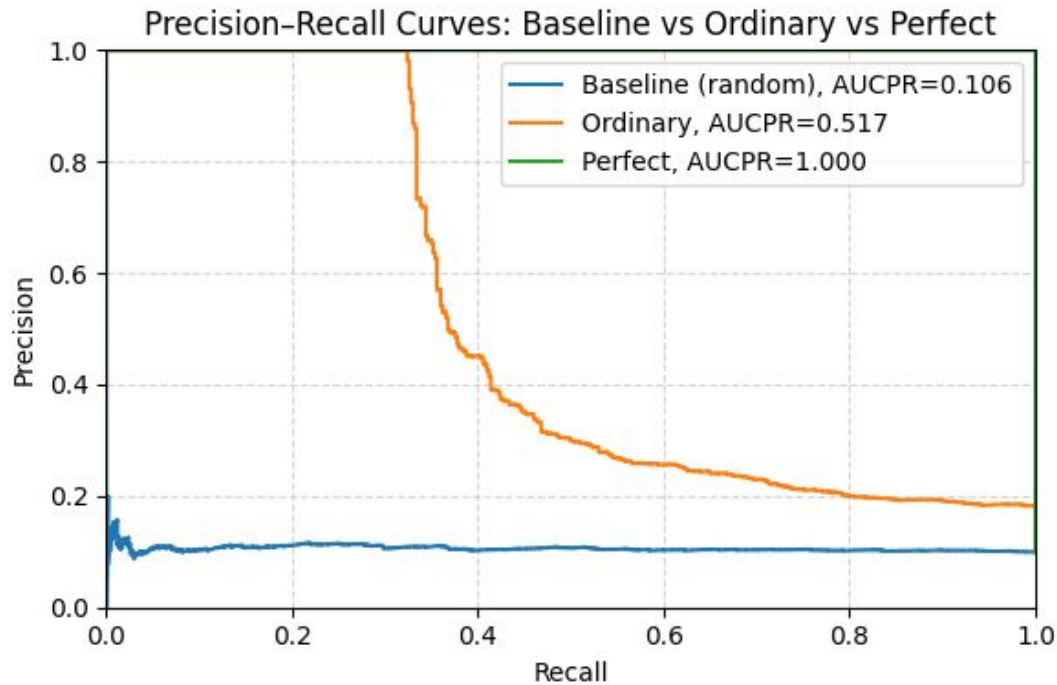


This trade-off creates multiple (Precision, Recall) pairs

AUCPR



AUC = Area Under Curve

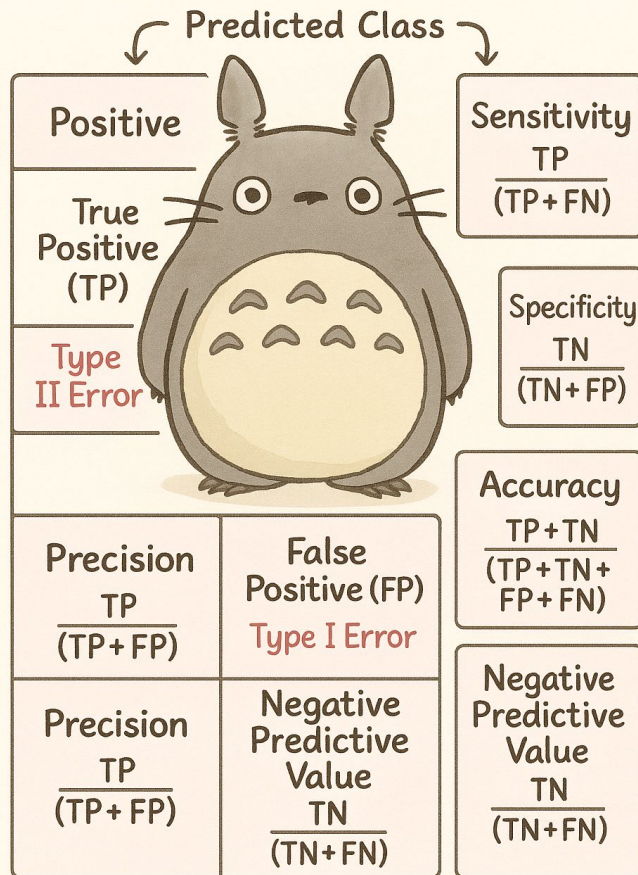


Choosing the Right Metric



- Regression $\rightarrow$ MSE, MAE, RMSE
- **Balanced** classification $\rightarrow$ Accuracy, F1
- **Imbalanced** classification $\rightarrow$ Precision, Recall, F1, AUCPR
- Unsupervised tasks $\rightarrow$ Normalized Mutual Information (later lectures)

Metrics





Bias - Variance Tradeoff



Mock Interview



Interviewer: “Your model gets 99% on training data and 80% on validation. What do you try first?”

- A) A bigger model
- B) Train longer
- C) More data or stronger regularization
- D) Add more features

30 seconds, no talking.

Bias–Variance Decomposition



Bias

- **Error** caused by **simplifying assumptions** in the model, leading to a **systematic difference** between prediction and ground truth

Symptom

- Model is too simple to capture the true relationship
- Leads to underfitting
- Example: Using a straight line to fit a curved pattern

Bias–Variance Decomposition



Variance

- **Error** caused by a model's **sensitivity** to small fluctuations in **the training data**, leading to **inconsistent predictions** across different datasets.

Symptom

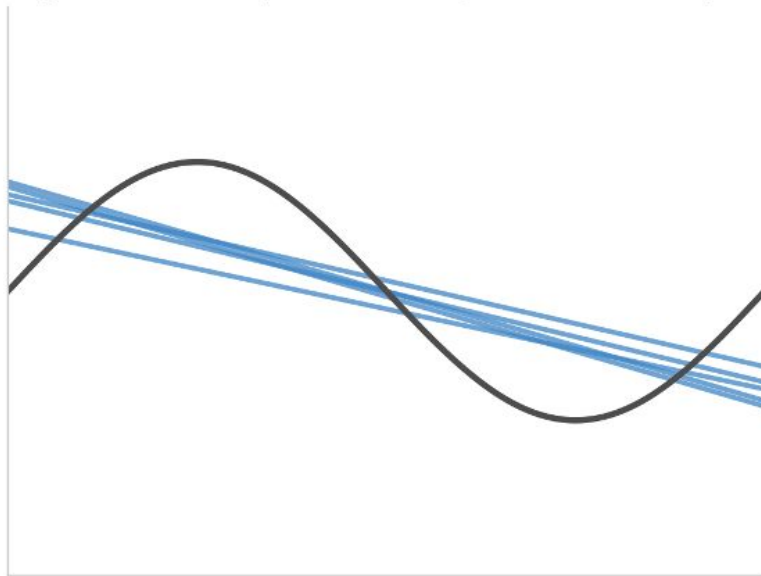
- Model is too complex, changes a lot with small noises in data
- Leads to overfitting

Same Model, 5 Different Training Sets



Simple model (straight line)

High bias: every fit is wrong the same way

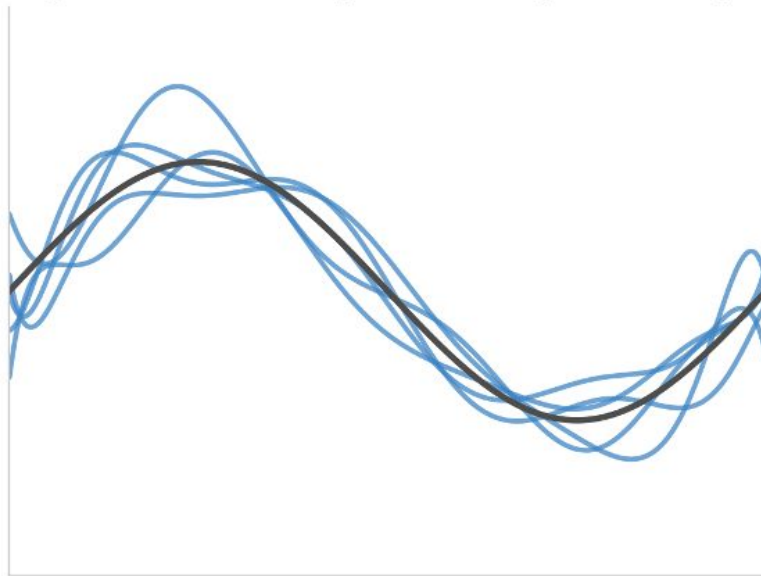


— True pattern

— 5 fits, each on a new training sample

Complex model (degree-9 polynomial)

High variance: every fit is wrong differently



— True pattern

— 5 fits, each on a new training sample

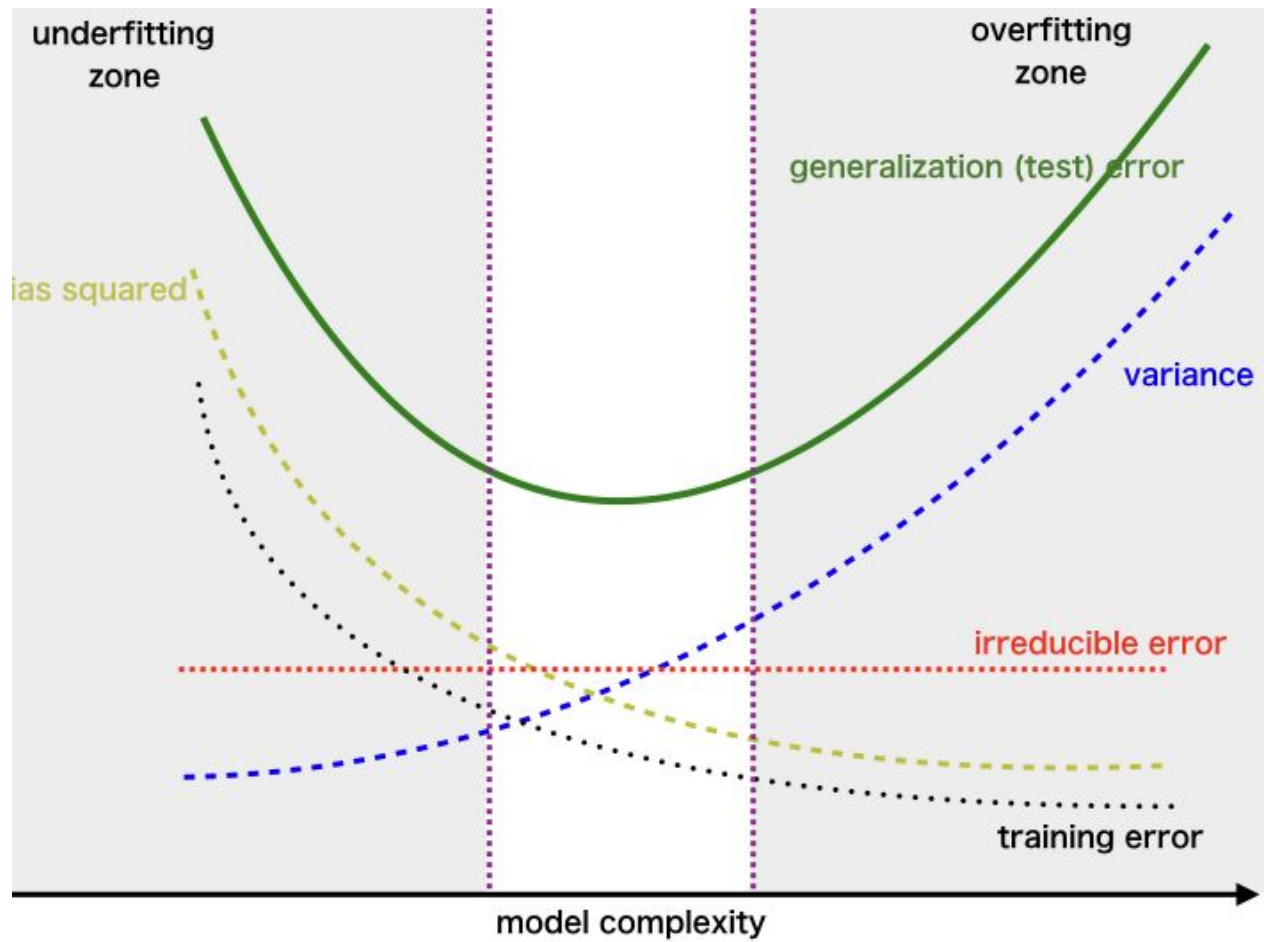


Bias–Variance Tradeoff

Generalization Error = $\text{Bias}^2 + \text{Variance} + \text{Noise}$

- Generalization Error == Expected Test Error

Goal = balance between **Bias** (too simple → underfit) and **Variance** (too complex → overfit)



Bias–Variance in Industry



Nobody computes bias or variance. You read two numbers:

- Train $\gg$ Val, complex model $\rightarrow$ **high variance** $\rightarrow$ more data, regularize
- Train $\approx$ Val, both bad $\rightarrow$ **high bias** $\rightarrow$ bigger model

How much data you have decides which one you fight:

- 300 support tickets $\rightarrow$ **variance** $\rightarrow$ keep the model simple
- Billions of ad clicks $\rightarrow$ **bias** $\rightarrow$ models keep getting bigger

Bias vs Variance — Key Takeaways



Opposite failure modes

- High Bias → Underfitting, model too rigid
- High Variance → Overfitting, model too sensitive

Tradeoff is unavoidable

- Reducing bias usually increases variance
- Reducing variance usually increases bias

What we really want

- Minimize expected test error (generalization error)
- Balance both terms + accept some irreducible noise



Model Selection Strategies

Why Model Selection?



We have mastered:

- How to split data 
- How to measure performance 
- Why bias & variance matter 

Next challenge:

- Choosing **the best model** among candidates

Cross-Validation for Model Choice



Not just for evaluation → also for model selection

Use **k-fold CV** to estimate performance

Pick model with best **average** validation score

Hyper-parameter Tuning



Models often have parameters not learned directly

- Regularization strength (α)
- Decision tree depth
- Neural network depth

Strategies

- Grid search 
- Random search 

Regularization



Problem: overly complex models $\rightarrow$ high variance

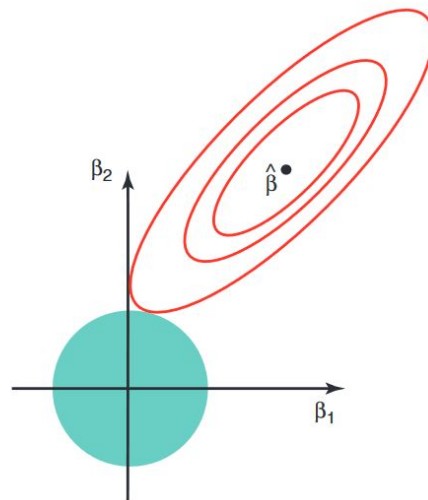
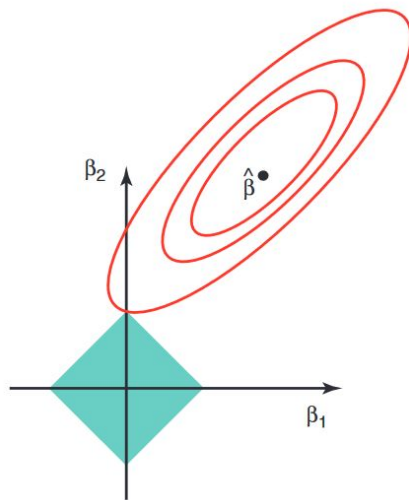
Solution: add penalty terms

- Ridge (L2): penalize large weights smoothly
- Lasso (L1): shrink some weights to zero $\rightarrow$ feature selection

Ridge vs Lasso (Geometry)



Lasso	Ridge
diamond constraint $\rightarrow$ corners	circular constraint $\rightarrow$ smooth shrinkage



Ensemble Methods



Combine multiple models to **reduce variance**

Examples:

- Bagging / Random Forests 
- Boosting (AdaBoost, XGBoost) 

Analogy: “wisdom of the crowd”

Practical Workflow



- Split data into Train / Val / Test
- Define candidate models + hyperparameters
- Use Cross-validation for fair comparison
- Regularize if variance too high